REQUIREMENT ANALYSIS

Minet Zambia

REQUIREMENT ANALYSIS

Requirement Analysis is the process of understanding what a system is supposed to do before development begins. It involves studying the problem, learning what users need, and deciding what features the software must have.

Requirement Analysis is important because it helps us avoid building the wrong system. When we take time to understand the problem, we make better decisions and create software that solves real challenges.

Identifying the Problem

Before anything else, we must clearly know the problem the software is meant to solve. For a school registration system, the problem might be slow enrollment, misplaced student files, or long queues. If we don't identify the core problem, we may build a system that looks good but doesn't help anyone.

Understanding User Needs & Expectations

After identifying the problem, the next step is listening to users. Users are the people who will use the system students, parents, teachers, and administrators. In a school scenario, parents expect quick registration without paperwork. Teachers want to see student lists easily. The system should be built based on what each user needs and expects.

Understanding the Scope of the System

Scope refers to what will be included in the system and what will not. It helps us draw a clear boundary, so we don't add unnecessary features later. For example, if your system is a school registration platform, the scope may include online student registration, document uploads, and automatic admission numbers. It may not include things like fee payment or class timetables if they are not needed at this stage. When scope is clear, development becomes more focused and efficient rather than confusing or overloaded.

Identifying Constraints and Limitations

Every system has restrictions such as time, budget, available skills, or the type of technology that must be used. These limitations affect how the final system will be built. For instance, if a school only has basic internet access, the system must be lightweight and easy to run. If the project has a short deadline, only essential features should be prioritized first. Knowing these constraints early helps us set realistic expectations and prevents delays or failure.

Functional Requirement

A functional requirement explains what a system must be able to do. It focuses on the actual actions or tasks the software should perform for the user. In the case of a School Registration System, a functional requirement would be something like this:

The system should allow a student to register online by entering their personal details and submitting them to the school database. When describing functional requirements, we are basically describing the features of the software the things users will interact with directly.

Non-Functional Requirement

A non-functional requirement, on the other hand, describes how the system should behave rather than what it should do. It deals with the quality and performance of the system, such as speed, security, reliability, accessibility, or ease of use. Using the same School Registration System example, a non-functional requirement would read like this:

The registration platform should load quickly and respond within a few seconds to avoid frustrating users during peak registration times. Here, we are not describing a feature instead, we are explaining how well that feature must operate.

Practice Task

Choose any system you want for example a School Registration System, a calculator, or a to-do list app and write one short sentence that shows a functional requirement and another that shows a non-functional requirement.

The functional requirement should describe what the system must do, while the non-functional requirement should describe how well it should perform or behave.

REQUIREMENT GATHERING

1. Definition

Requirement Gathering is the stage where we collect information from users and stakeholders to understand what they want the system to do. It is about asking questions, observing processes, and gathering details that will later guide development. Without gathering requirements, we may build a system based on assumptions rather than real needs.

2. Why Requirement Gathering is Important

We gather requirements so that the final product matches user expectations. It helps us understand everyday challenges, workflows, and the results people expect from the system. When we engage users early, they feel involved, and we avoid re-doing work later. Good gathering leads to clear understanding, accurate features, and smoother development.

3. Ways Requirements Can Be Gathered

1. Interviews: Talk to users to learn their needs.
   Example: Interview parents and teachers about registration frustrations.

2. Observation: Watch how current processes happen.
   Example: Observe long queues and lost forms during school registration.

3. Questionnaires / Surveys: Ask multiple users to fill forms about their needs.
   Example: Send a survey to parents asking what features they want in the registration system.

4. Workshops / Focus Groups: Gather multiple users to discuss ideas together.
   Example: Organize a session with teachers, parents, and admin to brainstorm features.

4. Who Is Involved in Requirement Gathering

People involved are known as stakeholders anyone who will use the system or benefit from it. In a school registration example, parents want quick enrollment, teachers want easy class lists, and the admin wants organized data and fewer errors. Listening to all these groups gives a full picture of what the system should support.

5. Practice Task Question

Imagine you are building a School Registration System. Identify two stakeholders and explain how you would gather their requirements.

FEASIBILITY / VISIBILITY STUDY

A Feasibility or Visibility Study is the process of evaluating whether a proposed system can be successfully developed and implemented. It looks at the technical, operational, and economic aspects of the system to determine if the project is practical and worthwhile before starting development. Essentially, it answers the question: "Can we really build this system, and will it work as intended?"

Purpose of a Feasibility Study

The main goal is to avoid wasting time, money, and resources on a system that may fail. By conducting a feasibility study, we identify potential problems, assess risks, and determine whether the system can meet user needs.

Types of Feasibility

Feasibility can be looked at from several angles.

Technical feasibility examines whether the hardware, software, and technical skills exist to build the system.

Operational feasibility evaluates if the system will be accepted and used effectively by the intended users, such as teachers, parents, and students.

Economic feasibility checks whether the cost of building and maintaining the system is justified by the benefits, like saving time, reducing paperwork, and improving accuracy.

Practical Example (School Registration System)

Before starting the project, we might ask: Do we have computers, internet, and trained staff to handle an online registration system?

Can parents and students easily access it from home?

Will the cost of developing the system be less than the benefits of faster registration and fewer lost records? By answering these questions, we ensure the project is realistic and likely to succeed.

SOFTWARE DESIGN, UML, AND PROGRAMMING LANGUAGES IN THE SDLC

Software Development Lifecycle (SDLC)

Definition

The Software Development Lifecycle (SDLC) is a structured process that describes the steps involved in developing software, from the initial idea to the final product. It provides a roadmap to guide developers, designers, and stakeholders, ensuring the system meets user requirements, is delivered on time, and functions correctly.

Following SDLC reduces errors, prevents wasted effort, and ensures a high-quality software product.

Phases of SDLC



1. Requirement Analysis
This is the first step, where we study and understand what the system should do. Stakeholders, such as users, clients, and administrators, are consulted to gather functional and non-functional requirements. For example, in a School Registration System, this involves identifying the need for online student registration, tracking admission status, and generating reports.

2. Requirement Gathering
Once the problem and needs are identified, we collect detailed information on what the users want and how they expect the system to behave. Methods include interviews, surveys, observation, and focus group discussions. The goal is to have a complete and accurate set of requirements that the system must meet.

3. Design Phase
In this phase, we plan **how the system will work**. Software design includes architecture planning, defining modules, workflows, and data structures. Tools like **UML diagrams** (Use Case, Class, Sequence) are used to visualize the system and make it easier for developers to understand.

4. Development / Implementation
Here, the actual coding happens. Developers use chosen **programming languages** to build the system according to the design specifications. For example, a web-based school system might use React for the frontend and Node.js for the backend. Mobile versions could be built using Flutter and Dart.

5. Testing
After development, the system is tested to ensure it works as expected. Bugs, errors, and issues are identified and fixed. Testing ensures the system is reliable, secure, and performs well under different scenarios.

6. Deployment
Once tested, the system is deployed to users. This could be an online platform, mobile app, or desktop application. Deployment involves installation, configuration, and making sure the system is accessible to all intended users.

7. Maintenance
After deployment, ongoing support is required. Maintenance includes fixing bugs, updating features, and ensuring the system continues to meet user needs over time.

STEP BY STEP INTO SOFTWARE DEVELOPMENT LIFE CYCLE

1. Definition of Software Design

Software design is the process of planning how a system will work before any coding begins. It is like creating a blueprint for a building you don't start laying bricks until you know the shape, structure, and layout of the house. In software, design helps us understand how different parts of the system will interact, what components are needed, and how information will flow.

This step is important because it reduces confusion, prevents costly mistakes during development, and makes sure everyone on the team has a shared understanding of what will be built.

2. System Architecture

System architecture refers to the overall structure of a software system the big picture view of how everything fits together. It explains how the system's parts communicate, how data moves, and where different responsibilities lie. In simple terms, system architecture is like a map showing the main roads, buildings, and routes in a city.

There are different types of architecture depending on the kind of system being built. For example

Client–server architecture is commonly used for systems that require a frontend (what the user sees) and a backend (where processing happens).

Mobile app architecture focuses on how apps run on phones and interact with servers online.

Web architecture deals with browser-based systems where users access features through the internet. Understanding architecture helps developers design systems that are efficient, secure, and easy to maintain.

3. Modules and Components

A module is a small, independent part of a larger system that performs a specific function. Breaking a system into modules makes development easier because each part can be built, tested, and improved separately. This approach is like dividing a big task into smaller pieces so that each team member can focus on their section without interfering with others.

In a School Registration System, example modules may include

- ✓ Student Management module, where you store and update student information;
- ✓ A Registration module, where students can enroll or apply.
- ✓ Class Allocation module, which assigns students to classes
- ✓ Reports module, which generates summaries for administrators. When modules are clearly defined, the system becomes more organized, scalable, and easier to maintain.

4. Workflows and Processes

Workflows describe the step-by-step path a user takes to complete a task in the system. They help us understand how information should flow and how the system should respond at each stage. Designing workflows ensures that users will have a smooth experience and that the system supports their activities correctly.

For example, a typical Student Registration Flow in a School Registration System might start with a student opening the registration page. Next, they fill in personal details, upload necessary documents, and submit the form. The system then validates the information, stores it in the database, and notifies the school administration for approval. Once approved, the student receives a confirmation message. Mapping this workflow helps developers know which screens, validations, and database actions are required, making the design complete and user-friendly.

5. Data Structures and Database Planning

Data structures describe how information will be stored and organized inside a system. In software design, data structures help us understand what type of data the system will collect, how that data will be arranged, and how different pieces of information connect to each other. This planning is important because it affects the system's speed, accuracy, and ability to grow in the future.

Database planning is the process of deciding which tables the system needs, what fields (columns) each table should have, and how the tables relate to one another. This is like arranging files in a well-organized cabinet every document has a place, and everything is easy to find.

Database for a School Registration System

To make this practical, here is a simple example of database tables that could be used in a School Registration System

1. **Students Table**

Column Name	***Description***
student_id (PK)	Unique ID for each student
first_name	Student's first name
last_name	Student's last name
date_of_birth	Date of birth
gender	Gender of the student
address	Home address
guardian_name	Parent/guardian full name
guardian_contact	Phone number of guardian

2. **Registration Table**

Column Name	***Description***
registration_id (PK)	Unique ID for each registration
student_id (FK)	Connects to the Students table
academic_year	Year of registration (e.g., 2025)
status	Pending / Approved / Rejected
date_submitted	Date the student submitted the form

This table captures each registration attempt and allows the school to track progress.

Classes Table

Column Name	*Description*
class_id (PK)	Unique ID for each class
class_name	Name of the class (e.g., Grade 8A)
capacity	Maximum number of students allowed

3. **Class Allocation Table**

Column Name	*Description*
allocation_id (PK)	Unique allocation record
student_id (FK)	ID of the student assigned
class_id (FK)	ID of the assigned class
term	Term of the year (e.g., Term 1)

This table helps track which student belongs to which class.

---

4. **Users Table (Optional)**

For systems with login access.

Column Name	*Description*
user_id (PK)	Unique ID
username	Login name
password_hash	Encrypted password
role	admin/teacher/registrar/student

How This Supports Software Design

When these tables are created during the design phase, developers clearly understand:

- what information the system must store,
- how different parts of the system interact,
- what workflows connect to which tables,

- and how to structure forms, screens, and logic during development.

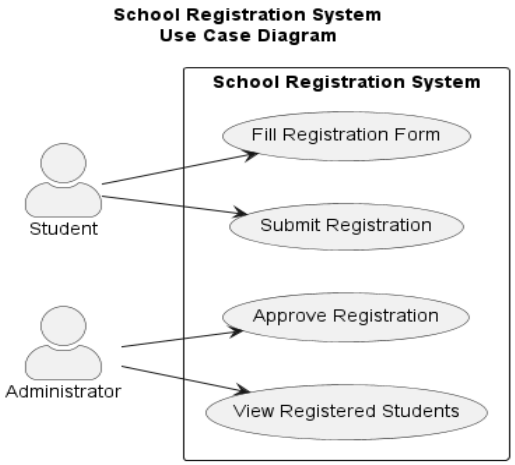
For example, when a student clicks **Submit Registration**, the system will save the data into the **Registration table**, link it with the **Students table**, and notify administrators.

UML DESIGN

1. Use Case Diagram

A Use Case Diagram shows the interaction between users (called actors) and the system. It focuses on what the users want to achieve rather than how the system works internally. This is usually the first UML diagram created because it clearly defines the system's main functions.

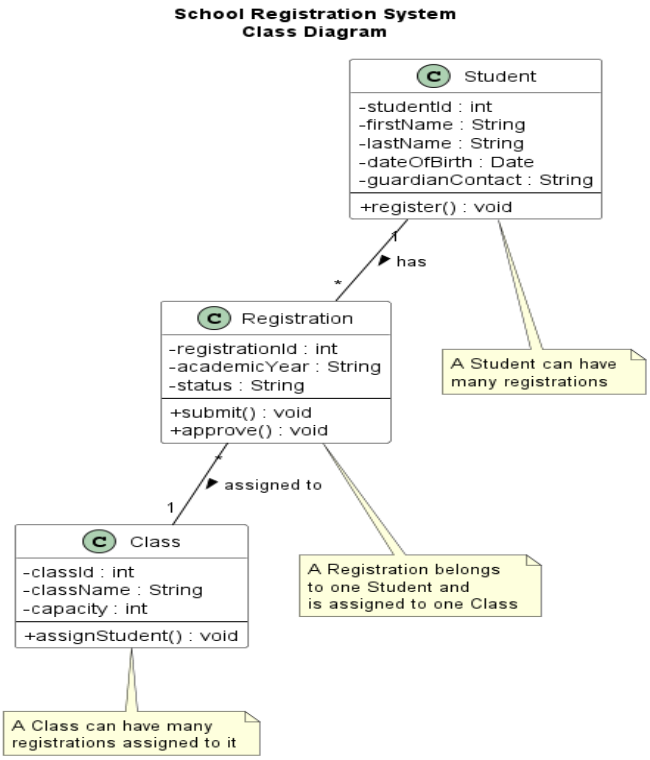
In a School Registration System, we identify the actors (like Students and Administrators) and show the actions they can perform. The goal is to understand who uses the system and what features they need.



2. Class Diagram

A Class Diagram shows the structure of the system. It focuses on the data (attributes) and the relationships between objects. This diagram acts like a blueprint of the database and backend structure.

It helps developers understand what objects exist in the system and how they are connected.



3. Sequence Diagram

A Sequence Diagram shows how different parts of the system interact over time. It focuses on the order of actions, starting from what the user does up to how the system responds. It is perfect for describing system workflows.

This diagram is useful for developers to understand the flow of logic.

